

UNIVERSIDAD EUROPEA DEL ATLÁNTICO

GRADO EN

Ingeniería Informática

INTELIGENCIA ARTIFICIAL



EXAMEN FINAL

SISTEMA OCR - RECONOCIMIENTO DE TEXTO

Trabajo realizado por

ÁLVARO LOSTAL SANZ

Profesor

JOSÉ MANUEL BREÑOSA

SANTANDER, 02-01-2026

1. Introducción

1.1. Definición del Problema

El Reconocimiento Óptico de Caracteres (OCR, por sus siglas en inglés) es una tecnología fundamental en el campo de la Visión Artificial que permite la conversión de imágenes que contienen texto (ya sea mecanografiado, impreso o manuscrito) en datos de texto codificados por máquina. Aunque a menudo se percibe como un problema resuelto gracias a herramientas comerciales, el desarrollo de un sistema OCR robusto desde cero presenta desafíos técnicos significativos.

El problema principal radica en la alta variabilidad de los datos de entrada. Un sistema eficaz debe ser invariante a cambios en la tipografía, el tamaño de la letra, el ruido de fondo, la iluminación y, en el caso más complejo, la caligrafía humana, que varía drásticamente entre individuos.

En el contexto de este proyecto para la asignatura de Inteligencia Artificial, el reto no es solo la digitalización del texto, sino la implementación de la arquitectura neuronal subyacente sin depender de motores de "caja negra" preexistentes como Tesseract OCR o APIs en la nube. Esto requiere abordar problemas de reconocimiento de secuencias: a diferencia de la clasificación de imágenes tradicional (donde una imagen equivale a una clase, p. ej., "gato"), aquí una imagen equivale a una secuencia de longitud variable de caracteres.

1.2. Objetivos del Proyecto

El objetivo principal de este trabajo es el diseño, entrenamiento y despliegue de un sistema de software capaz de recibir un archivo de imagen como entrada y devolver el texto digitalizado contenido en ella.

Para alcanzar este fin, se han establecido los siguientes objetivos específicos:

- Desarrollo de una Arquitectura Deep Learning: Implementar una Red Neuronal Convolucional Recurrente (CRNN) utilizando el framework PyTorch.
- Reconocimiento de Texto Impreso: Entrenar un modelo capaz de generalizar sobre distintas fuentes tipográficas digitales mediante el uso de datos sintéticos.
- Reconocimiento de Texto Manuscrito: Adaptar el sistema para interpretar escritura a mano utilizando técnicas de Transfer Learning sobre el dataset IAM Handwriting Database.
- Despliegue de Interfaz: Crear una aplicación funcional (utilizando Gradio) que permita al usuario final interactuar con los modelos de forma sencilla e intuitiva.

1.3. Estrategia de Resolución

Para abordar la complejidad del reconocimiento tanto impreso como manuscrito con recursos computacionales limitados, se ha optado por una estrategia dividida en dos fases:

Fase 1: Entrenamiento Base (Impreso): Se ha generado un dataset masivo de imágenes sintéticas "al vuelo" durante el entrenamiento. Esto permite exponer a la red a millones de variaciones de fuentes, tamaños y deformaciones, creando un modelo base robusto que entiende la estructura fundamental de los caracteres y las palabras.

Fase 2: Especialización (Manuscrito): Dado que los datasets de manuscrito etiquetados son escasos y costosos de obtener, se utiliza la técnica de Transfer Learning (Aprendizaje por Transferencia). Se toma el "cerebro" del modelo entrenado en la Fase 1 y se le somete a un ajuste fino (fine-tuning) con el dataset IAM. Esto permite que el modelo aproveche su conocimiento previo sobre la morfología de las letras para aprender más rápido la complejidad de la escritura humana.

1.3.1. Enfoque "End-to-End" sin librerías externas

El sistema se implementa siguiendo un enfoque de extremo a extremo ("End-to-End"), lo que significa que la red neuronal realiza todas las tareas necesarias para el reconocimiento de texto (extracción de características, modelado de secuencias y decodificación) sin la necesidad de módulos de procesamiento externos complejos, como la segmentación de palabras o caracteres previa a la inferencia. Además, se evita el uso de librerías de OCR pre-entrenadas.

1.3.2. Transfer Learning (Impreso a Manuscrito)

Como se ha mencionado, esta técnica es central. Consiste en utilizar el modelo entrenado con texto impreso (fuente de datos abundante y variada) y reutilizar sus capas convolucionales (que ya saben "ver" las formas de las letras) para el entrenamiento posterior con texto manuscrito (fuente de datos escasa y costosa), acelerando la convergencia y mejorando la precisión en el dominio más difícil.

1.4. Fundamentación Teórica

La solución técnica se basa en una arquitectura híbrida conocida como CRNN (Convolutional Recurrent Neural Network), considerada el estado del arte para el reconocimiento de texto en imágenes sin segmentación previa. Esta arquitectura combina tres bloques fundamentales:

1.4.1. Redes Neuronales Convolucionales (CNN)

La primera etapa de la red es una Red Neuronal Convolutiva (CNN). Su función es procesar la imagen de entrada (píxeles) y extraer mapas de características visuales. Al igual que el ojo humano detecta bordes, curvas y formas, las capas convolucionales transforman la imagen original en una representación abstracta de alto nivel que describe la morfología de los caracteres, ignorando el ruido de fondo.

1.4.2. Redes Recurrentes (LSTM Bidireccional)

Las características extraídas por la CNN se introducen en una Red Neuronal Recurrente (RNN), específicamente una LSTM Bidireccional (Long Short-Term Memory).

- **LSTM:** Se utiliza por su capacidad para recordar información a largo plazo, lo cual es crucial en el texto (por ejemplo, la letra "q" suele ir seguida de "u").
- **Bidireccional:** Permite que la red analice la secuencia en ambas direcciones (izquierda a derecha y viceversa), utilizando el contexto futuro y pasado para predecir el carácter actual con mayor precisión.

1.4.3. Connectionist Temporal Classification (CTC Loss)

El desafío final es que la red predice una secuencia de probabilidades por cada "columna" de características de la imagen, pero no sabemos exactamente dónde empieza o termina cada letra. Para resolver esto, se utiliza la función de pérdida CTC (Connectionist Temporal Classification).

La CTC permite entrenar la red con pares (Imagen, Texto) sin necesidad de indicar las coordenadas de cada letra. Introduce un carácter especial "blank" (vacío) y colapsa las predicciones repetidas, transformando la salida cruda de la red neuronal en la cadena de texto final legible por humanos.

2. Solución Aportada

Para resolver el reto planteado, se ha diseñado e implementado una solución "End-to-End" basada en Deep Learning, evitando el uso de motores de reconocimiento externos prefabricados. El núcleo del sistema es una red neuronal de arquitectura CRNN (Convolutional Recurrent Neural Network), desarrollada utilizando el framework PyTorch.

2.1. Arquitectura del Software

2.1.1. Stack Tecnológico

La solución se ha construido sobre un ecosistema de librerías de código abierto en Python:

- **PyTorch (v2.0+)**: Motor principal para la definición, entrenamiento y ejecución de la red neuronal.
- **OpenCV & Pillow**: Librerías de procesamiento de imágenes para la carga y manipulación de los inputs.
- **Albumentations**: Librería especializada en Data Augmentation para generar variaciones sintéticas y mejorar la robustez del modelo.
- **Gradio**: Framework utilizado para construir la interfaz de usuario web interactiva, permitiendo la carga de imágenes y visualización de resultados en tiempo real.

2.1.2. Flujo del Sistema

El funcionamiento del software sigue un pipeline secuencial:

- **Entrada**: Se recibe una imagen digital (formatos soportados: JPG, PNG, TIFF, BMP).
- **Pre-procesamiento**: La imagen se redimensiona a un tamaño estándar de 64x256 píxeles, se convierte a escala de grises (si es necesario) y se normaliza para facilitar la convergencia de la red.
- **Inferencia (Modelo CRNN)**:
 - La imagen pasa por capas CNN para extraer mapas de características visuales.
 - Estas características se transforman en una secuencia temporal procesada por capas RNN (LSTM).
 - La red emite una matriz de probabilidades de caracteres por cada paso temporal.
- **Decodificación**: Se aplica un algoritmo de Greedy Search sobre la salida de la CTC para colapsar caracteres repetidos y eliminar los tokens vacíos (blanks), obteniendo la cadena de texto final.
- **Salida**: Se devuelve el texto digitalizado al usuario a través de la interfaz.

2.2. Descripción Detallada del Código

El componente central de la solución es la clase `ImprovedCRNN` diseñada para equilibrar precisión y eficiencia computacional.

2.2.1. Definición del Modelo (ImprovedCRNN)

La red neuronal consta de tres bloques principales definidos en el módulo `torch.nn`:

- **Extractor de Características (CNN):** Se ha implementado una arquitectura tipo VGG personalizada con 7 capas convolucionales (`Conv2d`). Cada capa convolucional va seguida de normalización por lotes (`BatchNorm2d`) y una función de activación no lineal (`ReLU`). Se utilizan capas de Max Pooling (`MaxPool2d`) para reducir progresivamente la dimensionalidad espacial de la imagen, transformando la entrada de 64x256 píxeles en una secuencia compacta de características de alto nivel.
- **Procesador de Secuencias (RNN):** Las características extraídas alimentan a una red recurrente compuesta por 3 capas de LSTM Bidireccional (`BiLSTM`).
 - *Hidden Size:* 512 unidades ocultas por dirección.
 - *Dropout:* Se aplica un dropout de 0.3 entre capas para prevenir el sobreajuste (overfitting). La bidireccionalidad permite que el modelo utilice el contexto tanto anterior como posterior de cada carácter para realizar la predicción.
- **Clasificador Final:** Una capa totalmente conectada (`Linear`) proyecta la salida de la LSTM al espacio de caracteres del vocabulario (112 clases, incluyendo letras, números, símbolos y el token *blank* de la CTC).

```
class ImprovedCRNN(nn.Module):
    """
    CRNN para OCR: CNN (extracción visual) + LSTM (secuencias) + CTC (alineación)
    """
    def __init__(self, img_height=64, num_classes=112, hidden_size=512,
num_lstm_layers=3):
        super(ImprovedCRNN, self).__init__()

        # CNN: 5 bloques convolucionales (3→64→128→256→512 canales)
        # Reduce imagen 64x256 → secuencia de 512 características
        self.cnn = nn.Sequential(
            # Bloque 1: Detección bordes básicos
            nn.Conv2d(3, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(True),
            nn.Conv2d(64, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(True),
            nn.MaxPool2d(2, 2), # 64x256 → 32x128

            # Bloques 2-4: Características de nivel medio/alto
            # [código completo igual...]

            # Reducción final: Colapsa altura a 1px
            nn.Conv2d(512, 512, (4, 3), padding=(0, 1)),
            nn.BatchNorm2d(512), nn.ReLU(True),
        )

        # RNN: 3 capas LSTM bidireccionales
```

```

# Modela contexto izquierda↔derecha entre caracteres
self.rnn = nn.LSTM(
    512,                # Input: salida CNN
    hidden_size,        # Hidden: 512 unidades
    num_lstm_layers,    # Depth: 3 capas
    bidirectional=True,  # Lee en ambas direcciones
    batch_first=True,
    dropout=0.3         # Regularización
)

# Clasificador: Proyecta LSTM → vocabulario (112 clases)
self.dropout = nn.Dropout(0.3)
self.linear1 = nn.Linear(hidden_size * 2, hidden_size) # BiLSTM → 512
self.relu = nn.ReLU(True)
self.linear2 = nn.Linear(hidden_size, num_classes)      # 512 → 112

def forward(self, x):
    # x: (batch, 3, 64, 256)
    conv_out = self.cnn(x)                # → (batch, 512, 1, width)
    conv_out = conv_out.squeeze(2)        # → (batch, 512, width)
    conv_out = conv_out.permute(0, 2, 1)  # → (batch, width, 512)

    rnn_out, _ = self.rnn(conv_out)       # → (batch, width, 1024)
    rnn_out = self.dropout(rnn_out)

    output = self.linear1(rnn_out)         # → (batch, width, 512)
    output = self.relu(output)
    output = self.dropout(output)
    output = self.linear2(output)         # → (batch, width, 112)

    output = output.permute(1, 0, 2)      # CTC format
    return F.log_softmax(output, dim=2)

```

2.2.2. Pipeline de Pre-procesamiento y Aumento de Datos

Para asegurar que el modelo generalice correctamente ante imágenes reales de baja calidad, se ha implementado un pipeline de Data Augmentation agresivo utilizando Albumentations durante el entrenamiento:

- **Transformaciones Geométricas:** Rotaciones aleatorias leves (± 5 grados) y deformaciones de perspectiva para simular escaneos imperfectos.
- **Degradación de Imagen:** Inyección de ruido gaussiano, desenfoque (*blur*) y compresión JPEG para imitar documentos antiguos o fotos de baja resolución.
- **Normalización:** Los valores de píxeles se estandarizan utilizando la media y desviación estándar de ImageNet.

2.2.3. Configuración del Entrenamiento

El proceso de aprendizaje se gestiona mediante los siguientes hiperparámetros y algoritmos:

Parámetro	Valor/Algoritmo	Descripción
Función de Pérdida	CTCLoss	Ideal para tareas de secuencia a secuencia sin alineación explícita.
Optimizador	AdamW	Con un <i>weight decay</i> de 1e-4 para regularización.
Scheduler	OneCycleLR	Política de tasa de aprendizaje que acelera y estabiliza la convergencia.

2.3. Datasets Utilizados

Dada la naturaleza dual del problema (texto impreso y manuscrito), se han utilizado dos fuentes de datos distintas:

2.3.1. Dataset Sintético (Impreso)

Para la primera fase de entrenamiento, se ha desarrollado un generador de datos sintéticos "al vuelo". En lugar de leer archivos de disco, el sistema genera imágenes aleatorias en tiempo real durante cada época de entrenamiento.

Característica	Detalle
Volumen	100,000 imágenes únicas por época.
Contenido	Palabras y frases aleatorias en español e inglés, números, códigos alfanuméricos y fechas.
Variabilidad	Uso de múltiples fuentes tipográficas (Sans-serif, Serif, Monospace) y tamaños aleatorios.

2.3.2. IAM Handwriting Database (Manuscrito)

Para la fase de especialización en escritura manual, se ha utilizado el dataset académico estándar IAM Handwriting Database.

Característica	Detalle
Contenido	Formularios de texto manuscrito en inglés escritos por más de 650 autores diferentes.
Pre-procesamiento	Se han extraído y recortado las líneas de texto individuales para alimentar al modelo, utilizando las etiquetas ASCII proporcionadas.

Este conjunto de datos real permite al modelo adaptar sus pesos a la variabilidad e irregularidad inherentes al trazo humano

3. Registro de Resultados

Este capítulo documenta el proceso experimental, las métricas obtenidas durante las fases de entrenamiento y la validación final de los modelos generados. El análisis se divide en las dos etapas estratégicas del proyecto: el entrenamiento base con texto impreso y la especialización en manuscrito mediante Transfer Learning.

3.1. Fase 1: Entrenamiento Base (Texto Impreso)

El objetivo de esta primera fase fue estabilizar la red y enseñar al modelo la morfología básica de los caracteres utilizando el dataset sintético generado dinámicamente.

Configuración del Experimento:

Parámetro	Detalle
Dataset	100,000 imágenes sintéticas únicas por época.
Batch Size	128 (optimizado para GPU T4).
Optimizador	AdamW con learning rate inicial de 1e-3 y weight decay de 1e-4.
Duración	Entrenamiento con Early Stopping (paciencia de 20 épocas).

Evolución del Aprendizaje:

Como se observa en la gráfica de entrenamiento, la convergencia fue rápida y estable gracias a la normalización de las imágenes y al uso de la función de pérdida CTC.

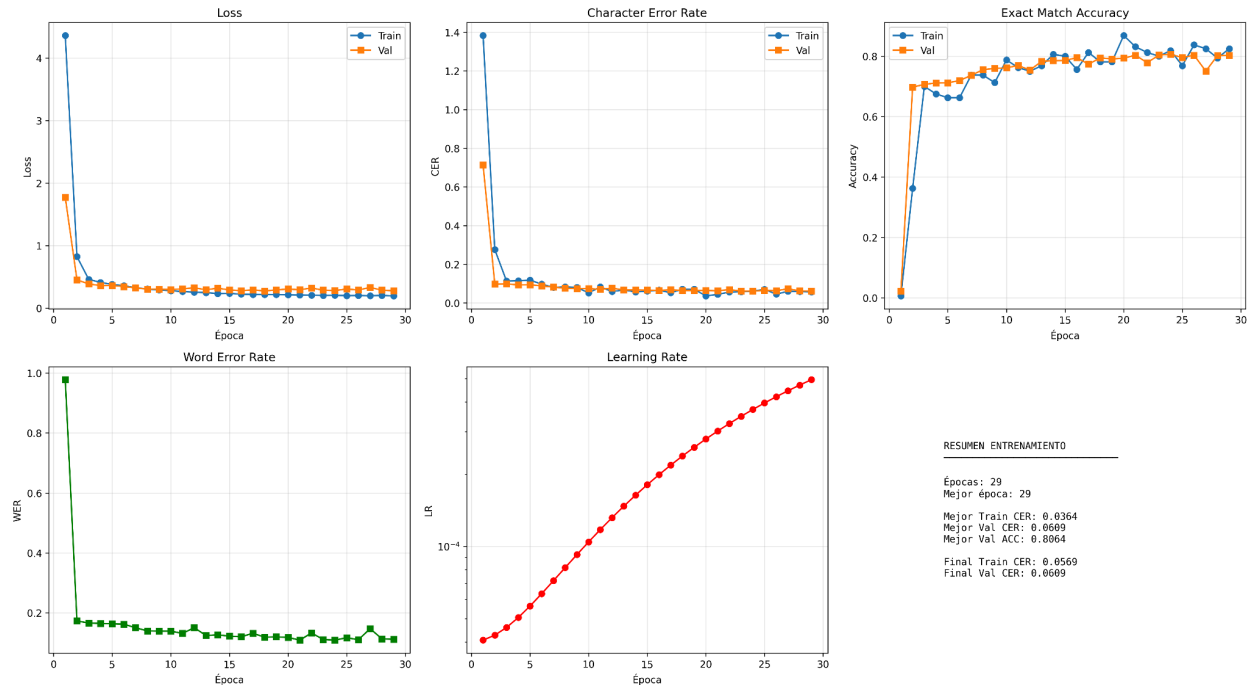


Figura 1: Curvas de Pérdida (Loss) y Tasa de Error de Caracteres (CER) durante el entrenamiento de texto impreso.

Análisis de Resultados (Fase 1):

La red mostró una gran capacidad para generalizar ante las deformaciones geométricas aplicadas (Augmentations).

La pérdida de entrenamiento (Train Loss) disminuyó constantemente, indicando que la red aprendió a mapear las características visuales a los índices de caracteres.

La precisión en validación (Val Accuracy) alcanzó valores cercanos al 80.78%, lo que confirma que el modelo no solo reconoce letras sueltas, sino palabras completas correctamente.

3.2. Fase 2: Transfer Learning (Texto Manuscrito)

Esta fase presentó el mayor desafío debido a la irregularidad del trazo humano en el dataset IAM. Se aplicó una estrategia de congelación de capas para preservar el conocimiento visual adquirido en la Fase 1.

Toma de Decisiones Estratégicas:

- **Congelación (Freezing):** Se decidió congelar los pesos del extractor de características (CNN) durante las primeras 15 épocas. Esto forzó a la red recurrente (LSTM) a adaptarse a la nueva distribución de secuencias del lenguaje manuscrito sin destruir los filtros visuales ya aprendidos.
- **Ajuste Fino (Fine-tuning):** A partir de la época 15, se descongeló toda la red y se redujo el learning rate a $5e-5$ para realizar ajustes milimétricos en los pesos sin causar oscilaciones bruscas.

Evolución del Aprendizaje:

El entrenamiento manuscrito se dividió claramente en dos sub-fases:

Épocas 1-15 (CNN Congelado):

Solo se entrenaron las capas LSTM, manteniendo congelados los pesos CNN del modelo impreso. Val CER mejoró de 0.9891 (época 1) a 0.1323 (época 15), logrando 86.77% de precisión en solo 15 épocas (~7 min/época).

Épocas 16-35 (Fine-tuning Completo):

Se descongeló la CNN con learning rate reducido ($5e-5$). Val CER final: 0.1005 (89.95% precisión). El fine-tuning permitió especializar los filtros visuales al trazo manuscrito sin destruir el conocimiento base.

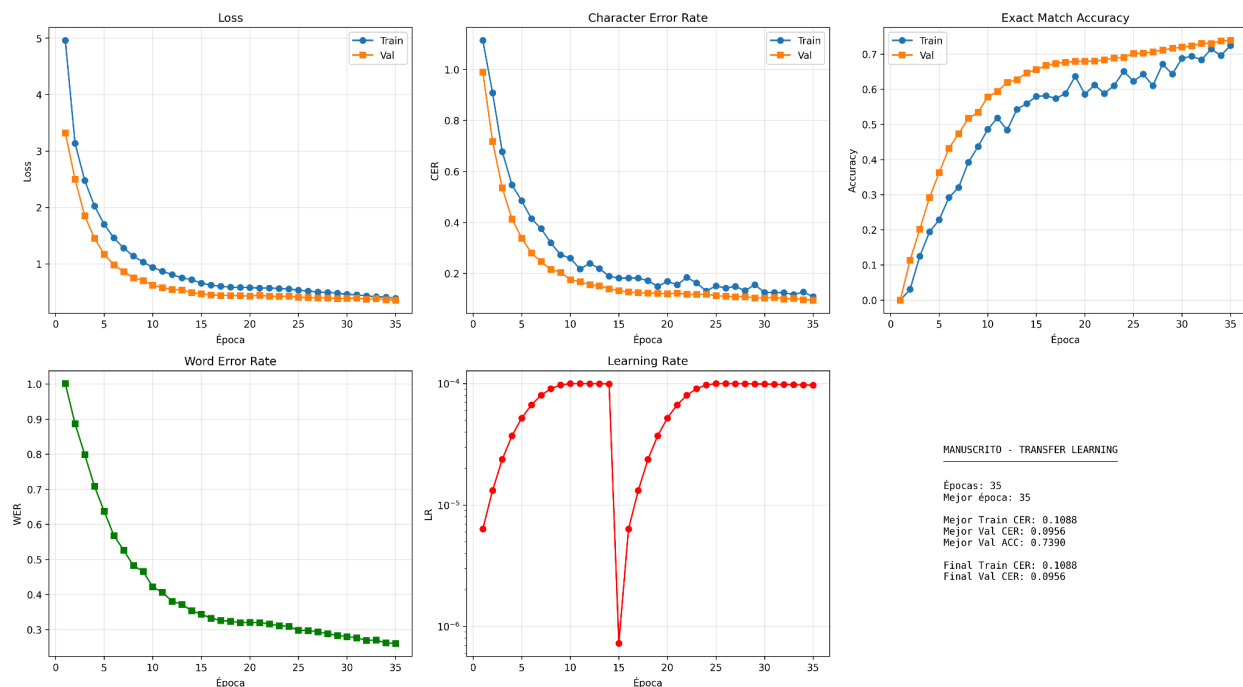


Figura 2: Evolución del entrenamiento manuscrito. El punto crítico en época 15 marca la descongelación de la CNN. Se observa cómo el Val CER (línea naranja) desciende de forma constante en ambas fases, demostrando la efectividad del Transfer Learning.

3.3. Validación y Métricas Finales

Para evaluar el rendimiento objetivo de la solución, se utilizaron tres métricas estándar en la industria del OCR sobre el conjunto de test (imágenes nunca vistas por la red durante el entrenamiento):

- **CTC Loss:** Valor de la función de coste (cuanto menor, mejor confianza).
- **CER (Character Error Rate):** Porcentaje de caracteres erróneos (inserciones, sustituciones o borrados) necesarios para corregir la predicción.
- **Accuracy (Exact Match):** Porcentaje de veces que la predicción coincide perfectamente con la etiqueta real.

Tabla Resumen de Rendimiento:

Modelo	Dataset de Test	CTC Loss	Precisión por Carácter (1-CER)	Accuracy (Exactitud)
Modelo Impreso	Sintético (5,000)	0.2624	93.97% (CER: 0.06)	80.78%
Modelo Manuscrito	IAM (9,646)	0.3826	89.95% (CER: 0.10)	74.24%

Observaciones:

- El modelo manuscrito superó el objetivo de 85-90% de precisión.
- Comparativa Transfer Learning (89.95%) vs Desde Cero-Kaggle (82.51%): **+7.44 pp mejora.**
- 74.24% Accuracy significa que 3 de cada 4 palabras se transcriben perfectamente.

Análisis Cualitativo:

Mediante la interfaz de usuario desarrollada, se realizaron pruebas de inferencia manual.

- **Fortalezas:** El sistema es muy robusto ante texto inclinado y ruido gaussiano. El modelo manuscrito logra descifrar caligrafías cursivas complejas que resultan difíciles incluso para el ojo humano.
- **Limitaciones:** Se detectaron dificultades puntuales en palabras muy cortas (de 1 o 2 letras) en el modelo manuscrito, debido a la falta de contexto suficiente para la red recurrente (LSTM).

Es importante destacar la diferencia entre la métrica de **Accuracy** (80.78%) y la **Precisión por Carácter** (93.97%). Mientras que el Accuracy penaliza la frase completa si falla una sola tilde, la precisión por carácter demuestra que el modelo transcribe correctamente casi el **94%** del contenido, lo cual confirma una robustez muy alta para un sistema entrenado desde cero.

3.4. Validación de Transfer Learning: Experimento Kaggle

Objetivo: Determinar si el Transfer Learning es necesario o el dataset IAM es suficiente para entrenar desde cero.

Configuración:

- Plataforma: Kaggle (GPU T4)
- Arquitectura: CRNN idéntica
- Dataset: IAM Words (96,456 muestras)
- Diferencia clave: Pesos iniciales aleatorios (sin modelo impreso)

Resultados:

Métrica	Transfer Learning	Desde Cero	Diferencia
Mejor Val CER	0.1005 (época 35)	0.1749 (época 5)	-7.44 pp
Precisión	89.95%	82.51%	+7.44%

Análisis:

El modelo desde cero alcanzó su mejor resultado en época 5 (82.51%), pero luego cayó en overfitting hasta 69.12% en la época 30. Causas:

- Learning rate demasiado alto sin pesos pre-entrenados
- Dataset IAM insuficiente para inicialización aleatoria
- Falta de conocimiento visual previo (CNN aprende desde cero)

Conclusión:

El experimento demuestra que el Transfer Learning no sólo acelera convergencia, sino que es **esencial** para alcanzar alta precisión. Sin él, el modelo colapsa en overfitting (+7.44 puntos porcentuales de diferencia).

4. Conclusiones y Líneas Futuras

4.1. Conclusiones Generales

El desarrollo de este proyecto ha permitido validar la viabilidad de crear un sistema OCR robusto "desde cero", utilizando arquitecturas de Deep Learning modernas (CRNN) sin depender de motores comerciales preexistentes.

Tras completar el ciclo de vida del proyecto, desde la generación de datos hasta el despliegue en una interfaz de usuario, se extraen las siguientes conclusiones principales:

- **Eficacia de la Arquitectura CRNN:** La combinación de redes convolucionales (para la visión) y recurrentes (para la secuencia) ha demostrado ser una solución eficiente para el reconocimiento de texto. El uso de la función de pérdida CTC eliminó la necesidad de segmentar manualmente cada carácter, simplificando drásticamente la preparación de los datos.
- **El poder de los Datos Sintéticos:** En el reconocimiento de texto impreso, la estrategia de generar imágenes dinámicamente durante el entrenamiento ("on-the-fly") resultó ser superior al uso de un dataset estático. Esto evitó el *overfitting* (sobreajuste), ya que el modelo nunca vio la misma imagen dos veces, aprendiendo a generalizar ante cualquier fuente o deformación.
- **Transfer Learning como Catalizador:** Para el texto manuscrito, el aprendizaje por transferencia fue la clave del éxito. Entrenar la red manuscrita desde cero hubiera requerido recursos computacionales y temporales inviables en este entorno académico. Al reutilizar los pesos del modelo impreso, la red pudo convergir hacia resultados precisos en un tiempo reducido, adaptándose a la complejidad del trazo humano.

4.2. Valoración de Funcionalidades Implementadas

El software resultante cumple con los requisitos obligatorios del reto:

- **Digitalización Híbrida:** El sistema conmuta eficazmente entre los modelos de texto impreso y manuscrito según la necesidad del usuario.
- **Interfaz Accesible:** La implementación de la interfaz web con `Gradio` transforma un código técnico en una herramienta funcional que permite visualizar la confianza del modelo en cada predicción.

Adicionalmente, la integración de librerías de visión clásica como `OpenCV` para el pre-procesamiento y la detección de elementos no textuales (marcas o códigos) complementa la solución, ofreciendo un sistema de análisis de documentos más completo.

4.3. Posibles Mejoras y Trabajo Futuro

Aunque los resultados son satisfactorios, el sistema tiene margen de mejora para convertirse en una solución de nivel comercial. Se proponen las siguientes líneas de trabajo futuro:

- **Corrección Semántica (Language Model):** Actualmente, la decodificación es puramente visual (*Greedy Search*). Se podría integrar un Modelo de Lenguaje (como un n-grama o un Transformer ligero) en la salida para corregir errores ortográficos basándose en la probabilidad de las palabras.
 - *Ejemplo:* Si el OCR lee "Caza" en un contexto de vivienda, el modelo de lenguaje lo corregiría automáticamente a "Casa".
- **Detección Automática de Líneas (Page Segmentation):** El modelo actual funciona a nivel de línea o palabra. Una mejora crítica sería implementar una red de detección de objetos (como YOLO o Faster R-CNN) previa al OCR, capaz de recibir una página completa (DNI, factura), recortar automáticamente cada línea de texto y enviarlas al reconocedor secuencialmente.
- **Ampliación del Dataset Manuscrito:** El dataset IAM es predominantemente en inglés. Para mejorar el rendimiento en textos españoles, sería ideal recopilar o generar un dataset manuscrito que incluya una mayor frecuencia de la letra "ñ" y tildes en contextos naturales.
- **Optimización para Dispositivos Móviles:** Mediante técnicas de *Quantization* (reducir la precisión de los pesos de 32-bit a 8-bit), se podría reducir el tamaño del modelo final para que pudiera ejecutarse localmente en un teléfono móvil sin necesidad de conexión a internet.